

2026-06-10 算法专题

新题：LC_0035 搜索插入位置 · LC_0074 搜索二维矩阵

复习：LC_0041 缺失的第一个正数 (R4) · LC_0238 除自身以外数组的乘积 (R4)

LC_0035 搜索插入位置

题目

给定一个排序数组和一个目标值，在数组中找到目标值，并返回其索引。如果目标值不存在于数组中，返回它将会被按顺序插入的位置。

请必须使用 $O(\log n)$ 的时间复杂度。

算法核心

标准二分查找，循环不变量 `[left, right]` 闭区间。当 `left > right` 退出循环时，`left` 就是目标值应该插入的位置。

代码

```
package algo.y2026.M06.d10;

public class LC_0035_SearchInsertPosition {

    public int searchInsert(int[] nums, int target) {
        if (nums == null || nums.length == 0) return 0;

        int left = 0, right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }
}
```

执行过程

```
nums = [1, 3, 5, 6], target = 2

初始: left=0, right=3

Step 1: mid = 0 + (3-0)/2 = 1
        nums[1] = 3 > 2 → right = mid - 1 = 0

Step 2: mid = 0 + (0-0)/2 = 0
        nums[0] = 1 < 2 → left = mid + 1 = 1

Step 3: left=1 > right=0 → 终止
        返回 left = 1
```

插入位置为索引 1，得到 [1, ****2****, 3, 5, 6] ✓

复杂度

维度	值
时间复杂度	O(log n)
空间复杂度	O(1)

Java 语法实现

- **防溢出**: `mid = left + (right - left) / 2` 代替 `(left + right) / 2`，避免大数相加溢出
- **闭区间写法**: `while (left <= right)`，对应 `left = mid + 1` 和 `right = mid - 1`
- **返回值含义**: 退出时 `left` 天然就是插入位置（不变量: `nums[left-1] < target < nums[left]`）
- **判空**: `nums == null || nums.length == 0` 统一返回 0

LC_0074 搜索二维矩阵

题目

编写一个高效的算法来判断 m x n 矩阵中，是否存在一个目标值。该矩阵具有如下特性：

- 每行中的整数从左到右按升序排列
- 每行的第一个整数大于前一行的最后一个整数

算法核心

将二维矩阵「拉直」成一维数组，用标准二分查找。下标映射： - 行: `row = mid / n` - 列: `col = mid % n`

代码

```

package algo.y2026.M06.d10;

public class LC_0074_SearchA2DMatrix {

    public boolean searchMatrix(int[][] matrix, int target) {
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) return false;

        int m = matrix.length, n = matrix[0].length;
        int left = 0, right = m * n - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            int row = mid / n;
            int col = mid % n;
            int val = matrix[row][col];

            if (val == target) {
                return true;
            } else if (val < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return false;
    }
}

```

执行过程

```

matrix = [
    [1, 3, 5, 7],
    [10, 11, 16, 20],
    [23, 30, 34, 60]
], target = 3

m = 3, n = 4, left = 0, right = 11

Step 1: mid = 0 + (11-0)/2 = 5
        row = 5/4 = 1, col = 5%4 = 1
        matrix[1][1] = 11 > 3 → right = 4

Step 2: mid = 0 + (4-0)/2 = 2
        row = 2/4 = 0, col = 2%4 = 2
        matrix[0][2] = 5 > 3 → right = 1

Step 3: mid = 0 + (1-0)/2 = 0
        row = 0/4 = 0, col = 0%4 = 0
        matrix[0][0] = 1 < 3 → left = 1

Step 4: mid = 1 + (1-1)/2 = 1
        row = 1/4 = 0, col = 1%4 = 1
        matrix[0][1] = 3 == 3 → return true ✓

```

复杂度

维度	值
时间复杂度	$O(\log(m \times n))$
空间复杂度	$O(1)$

Java 语法实现

- 多重判空: `matrix == null || matrix.length == 0 || matrix[0].length == 0` , 缺一不可
- 列数赋值: `int n = matrix[0].length` 注意是 `matrix[0]` 不是 `matrix`
- 三目定位: `mid / n` 得行号, `mid % n` 得列号 (整数除法截断特性)
- 边界: `right = m * n - 1` 因为一维下标从 0 开始

LC_0041 缺失的第一个正数

题目

给你一个未排序的整数数组 `nums` , 请你找出其中没有出现的最小的正整数。

要求时间复杂度 $O(n)$ 且空间复杂度 $O(1)$ 。

算法核心

原地哈希: 利用数组本身作为哈希表, 将每个数 `nums[i]` 放到它应该在的位置 `nums[i] - 1` 上。二次扫描找到第一个 `nums[i] != i + 1` 的位置, 返回 `i + 1`。

代码

```
package algo.y2026.M06.d03;

public class LC_0041_FirstMissingPositive {

    public int firstMissingPositive(int[] nums) {
        if (nums == null || nums.length == 0) return 1;

        int n = nums.length;
        for (int i = 0; i < n; i++) {
            while (nums[i] >= 1 && nums[i] <= n
                    && nums[nums[i] - 1] != nums[i]) {
                swap(nums, i, nums[i] - 1);
            }
        }

        for (int i = 0; i < n; i++) {
            if (nums[i] != i + 1) return i + 1;
        }
        return n + 1;
    }

    private void swap(int[] nums, int a, int b) {
        int tmp = nums[a];
        nums[a] = nums[b];
        nums[b] = tmp;
    }
}
```

```
        nums[a] = nums[b];
        nums[b] = tmp;
    }
}
```

执行过程

```
nums = [3, 4, -1, 1], n = 4

=== 第一遍：原地哈希 ===

i = 0: nums[0] = 3, 应放 idx = 2
      nums[2] = -1 ≠ 3 → swap(0, 2) → [-1, 4, 3, 1]
      nums[0] = -1 < 1 → 跳过

i = 1: nums[1] = 4, 应放 idx = 3
      nums[3] = 1 ≠ 4 → swap(1, 3) → [-1, 1, 3, 4]
      nums[1] = 1, 应放 idx = 0
      nums[0] = -1 ≠ 1 → swap(1, 0) → [1, -1, 3, 4]
      nums[1] = -1 < 1 → 跳过

i = 2: nums[2] = 3, 应放 idx = 2
      nums[2] = 3 ✓ → 跳过

i = 3: nums[3] = 4, 应放 idx = 3
      nums[3] = 4 ✓ → 跳过

数组变为 [1, -1, 3, 4]

=== 第二遍：扫描缺失 ===

i = 0: nums[0] = 1 == i + 1 ✓
i = 1: nums[1] = -1 ≠ 2 → return 2 ✓
```

复杂度

维度	值
时间复杂度	O(n) — 每个元素最多被交换一次
空间复杂度	O(1) — 原地交换

注意：暴力解法（逐个检查 1..n+1 是否存在）时间复杂度 O(n²)，空间 O(1)。

Java 语法实现

- 循环不 `i++`： `while` 内交换后 `nums[i]` 变成新值，需继续检查，所以用 `while` 而非 `if`，不移动 `i`
- `swap` 条件三要素：值 $\in [1, n]$ && 目标位 $\neq$ 当前值（防止死循环）
- 多值交换： `swap(nums, i, nums[i] - 1)` 中第二个参数依赖于 `nums[i]` 的当前值，注意顺序
- 返回 `n+1`：二次扫描全部命中 $\rightarrow$ 最小缺失是 `n + 1`

LC_0238 除自身以外数组的乘积

题目

给你一个整数数组 `nums`，返回数组 `answer`，其中 `answer[i]` 等于 `nums` 中除 `nums[i]` 之外其余各元素的乘积。

题目保证数组所有元素乘积在 32 位整数范围内，且不允许使用除法。

算法核心

两遍遍历：1. **左→右**： `answer[i]` 存 `nums[0..i-1]` 的乘积（左前缀） 2. **右→左**：用一个变量 `r` 滚动累乘右后缀，依次乘到 `answer[j]` 上

代码

```
package algo.y2026.M06.d03;

public class LC_0238_ProductExceptSelf {

    public int[] productExceptSelf(int[] nums) {
        if (nums == null || nums.length == 0) return new int[0];

        int len = nums.length;
        int[] answer = new int[len];

        answer[0] = 1;
        for (int i = 1; i < len; i++) {
            answer[i] = answer[i - 1] * nums[i - 1];
        }

        int r = 1;
        for (int j = len - 1; j >= 0; j--) {
            answer[j] = answer[j] * r;
            r = r * nums[j];
        }

        return answer;
    }
}
```

执行过程

```
nums = [1, 2, 3, 4]

=== 第一遍：左→右（构建左前缀）===

answer[0] = 1
i = 1: answer[1] = answer[0] * nums[0] = 1 * 1 = 1
i = 2: answer[2] = answer[1] * nums[1] = 1 * 2 = 2
i = 3: answer[3] = answer[2] * nums[2] = 2 * 3 = 6

answer = [1, 1, 2, 6]

=== 第二遍：右→左（乘上右后缀）===
```

```
r = 1
j = 3: answer[3] = 6 * 1 = 6    r = 1 * 4 = 4
j = 2: answer[2] = 2 * 4 = 8    r = 4 * 3 = 12
j = 1: answer[1] = 1 * 12 = 12  r = 12 * 2 = 24
j = 0: answer[0] = 1 * 24 = 24  r = 24 * 1 = 24

answer = [24, 12, 8, 6] ✓
```

验证： - $\text{answer}[0] = 2 \times 3 \times 4 = 24$ ✓ - $\text{answer}[1] = 1 \times 3 \times 4 = 12$ ✓ - $\text{answer}[2] = 1 \times 2 \times 4 = 8$ ✓
- $\text{answer}[3] = 1 \times 2 \times 3 = 6$ ✓

复杂度

维度	值
时间复杂度	$O(n)$ — 两遍线性遍历
空间复杂度	$O(1)$ — 除输出数组外只用了常数空间

Java 语法实现

- **复用 answer 数组**：左前缀直接存入 answer，省去额外数组
- **滚动变量 r**：存储当前右边所有元素的乘积，从右向左更新
- **乘积顺序**：`answer[j] = answer[j] * r` 先乘再更新 `r = r * nums[j]`，保证当前元素自身的值不会乘进去
- **判空**：`nums == null || nums.length == 0` 返回 `new int[0]` 而非 `null`